UNIVERSIDAD SERGIO ARBOLEDA

Escuela de Ingeniería y Ciencias Aplicadas

INFORME DE LABORATORIO

Taller Práctico 4: Procesamiento de Video en Algoritmos CUDA con GPU—Detección de Trayectoria de Asteroides

Autores:

Cesar Mora

Johan García

Asignatura métricas de rendimiento

Bogotá D.C. — abril de 2025

1. Objetivo del Laboratorio

El objetivo principal de este taller es desarrollar un algoritmo de procesamiento de video en tiempo casi real, capaz de detectar automáticamente la presencia de un asteroide y trazar su trayectoria dentro de un video de fondo estelar. Para ello se implementan dos versiones del algoritmo: una secuencial en CPU y otra paralelizada en GPU mediante CUDA con la librería Numba de Python, permitiendo comparar el rendimiento de ambas estrategias.

Los objetivos específicos son:

- Implementar el pipeline de detección de fondo por apilamiento de mediana (Median Stacking) de forma paralela en GPU.
- Aplicar sustracción de fondo, umbralización y operaciones morfológicas (erosión y dilatación) mediante kernels CUDA.
- Extraer la trayectoria del asteroide como una secuencia de centroides (x, y) por frame.
- Comparar tiempos de ejecución entre CPU y GPU para evidenciar la ventaja del cómputo paralelo.

2. Marco Teórico

2.1 Procesamiento de Video

Un video digital es una secuencia de imágenes (frames) capturadas a una tasa determinada de fotogramas por segundo (FPS). Cada frame es una matriz bidimensional de píxeles con valores de intensidad luminosa. En el contexto de visión artificial, el procesamiento de video consiste en aplicar operaciones matemáticas sobre estos arreglos para extraer información significativa, como la detección de objetos en movimiento.

2.2 Sustracción de Fondo (Background Subtraction)

La sustracción de fondo es una técnica que permite aislar objetos en movimiento de una escena con fondo estático. El principio fundamental consiste en construir un modelo del fondo y, para cada frame, calcular la diferencia absoluta:

$$I_n^{msMAIO} = | I^{cuadro} - Ifondo |$$

Los píxeles con valor superior a un umbral predefinido corresponden a regiones de cambio, es decir, al asteroide. Esta técnica es altamente paralelizable porque la operación de sustracción en cada píxel es completamente independiente.

2.3 Apilamiento de Mediana (Median Stacking)

Para construir la imagen de fondo, se utiliza el método de apilamiento de mediana. Si el asteroide aparece en una fracción minoritaria de los frames totales, el valor de la mediana temporal de cada píxel corresponde al fondo estático (estrellas y ruido), eliminando efectivamente el objeto en movimiento:

$$Ifondo(x, y) = mediana_t \{ I_t(x, y) \}$$

Este cálculo es computacionalmente costoso para videos de alta resolución, por lo que su implementación en GPU mediante kernels CUDA que procesan cada píxel en paralelo resulta fundamental.

2.4 Operaciones Morfológicas

Tras la sustracción y umbralización se aplican dos operaciones morfológicas clásicas sobre la máscara binaria:

- Erosión: elimina píxeles aislados producto del ruido, conservando solo regiones compactas.
- Dilatación: restaura el tamaño original del objeto detectado tras la erosión.

Ambas operaciones se implementan como kernels CUDA con un vecindario de 3×3 píxeles.

2.5 Computación Paralela con CUDA y Numba

CUDA (Compute Unified Device Architecture) es la plataforma de computación paralela de NVIDIA que permite ejecutar miles de hilos simultáneos en la GPU. En Python, la librería Numba proporciona el decorador @cuda.jit para compilar funciones Python como kernels CUDA en tiempo de ejecución.

Cada kernel se lanza con una configuración de grilla (grid) y bloques (blocks): el número total de hilos es grid_x × grid_y × block_x × block_y. Para imágenes de W×H píxeles, se asigna un hilo por píxel, logrando el máximo paralelismo posible.

3. Metodología

3.1 Configuración del Hardware

Parámetro	Valor / Descripción
Plataforma	Google Colaboratory (Colab)
GPU	NVIDIA Tesla T4 (16 GB GDDR6)
CUDA Cores	2560
CPU	Intel Xeon (2 vCPU, ~2.20 GHz)
RAM del sistema	12.7 GB
VRAM disponible	~15 GB

3.2 Configuración del Software

Parámetro	Valor / Descripción
Sistema operativo	Ubuntu 22.04 LTS (Colab)
Python	3.10.x
Numba	0.58.x (CUDA JIT)
OpenCV (cv2)	4.8.x (opencv-python-headless)
NumPy	1.24.x
Matplotlib	3.7.x
Driver CUDA	12.2
CUDA Toolkit	12.2

3.3 Videos de Prueba

Se utilizaron seis videos de prueba en formato MP4 proporcionados por el docente a través de Google Drive. Los videos simulan la observación del espacio desde un telescopio, con estrellas estáticas y un asteroide que atraviesa el campo visual a lo largo de los frames.

Video	Descripción	¿Contiene Asteroide?
video0	Fondo estelar puro, sin objeto en movimiento	No
video1	Asteroide con trayectoria diagonal ascendente	Sí
video2	Asteroide con trayectoria horizontal	Sí
video3	Asteroide de baja luminosidad	Sí
video4	Trayectoria curva / múltiples objetos	Sí
video5	Alta densidad estelar + asteroide	Sí

3.4 Descripción del Algoritmo Desarrollado

3.4.1 Pipeline General

El algoritmo sigue el siguiente pipeline en ambas versiones (CPU y GPU):

1. Leer todos los frames del video y convertirlos a escala de grises (float32).
2. Calcular la imagen de fondo mediante apilamiento de mediana píxel a píxel.
3. Para cada frame: aplicar blur 3×3, restar el fondo, umbralizar el resultado.
4. Aplicar erosión y dilatación morfológica de 3×3 para limpiar la máscara binaria.
5. Calcular el centroide de los píxeles activos. Si supera MIN_PIXELS, registrar posición.
6. Si se detectaron centroides, declarar “Existe Asteroide” y trazar la trayectoria.

3.4.2 Parámetros del Algoritmo

Parámetro	Valor / Descripción
THRESHOLD	10 — Valor mínimo de I_cambio para considerar un píxel como activo
MIN_PIXELS	5 — Número mínimo de píxeles activos para confirmar detección en un frame
KERNEL_SIZE	3 — Tamaño del kernel para blur y operaciones morfológicas
THREADS	(16, 16) — Hilos por bloque en la dimensión 2D (256 hilos totales/bloque)
Máx. frames soportados	512 — Límite del buffer local en kernel_median_stack

3.4.3 Kernels CUDA implementados

Se implementaron cinco kernels CUDA con Numba:

- kernel_blur: suavizado espacial 3×3 (media de vecindad). Reduce ruido de alta frecuencia.

- kernel_median_stack: calcula la mediana temporal por píxel. Usa insertion sort sobre un array local de hasta 512 frames.
- kernel_subtract: resta la imagen de fondo al frame actual y aplica umbralización binaria.
- kernel_erode: erosión morfológica 3×3. Elimina píxeles aislados por ruido.
- kernel_dilate: dilatación morfológica 3×3. Restaura el tamaño real del objeto detectado.

4. Resultados

4.1 Herramienta de Diagnóstico

Antes de ejecutar el algoritmo principal, se desarrolló una función de diagnóstico que calcula la imagen de fondo y muestra, para cada frame, el cambio máximo de intensidad y el número de píxeles que superan el umbral. Esto permite verificar la correcta construcción del fondo y calibrar los parámetros THRESHOLD y MIN_PIXELS.

```
# ──────────────────────────────────────────────
# Parámetros comunes  (ajusta si no detecta)
# ──────────────────────────────────────────────
THRESHOLD   = 20   # umbral de brillo para considerar 'cambio'
KERNEL_SIZE = 3    # tamaño del kernel morfológico / blur
MIN_PIXELS  = 5    # mínimo de píxeles activos para confirmar detección
```

Figura 2. *Salida del diagnóstico: cambio máximo y píxeles activos por frame..*

4.3 Detección Frame a Frame

Se muestran a continuación ejemplos representativos del proceso de detección en frames individuales: el frame original, la imagen de diferencia I_cambio, la máscara binaria tras umbralización y la máscara resultante tras erosión y dilatación con el centroide marcado.

Frame	Cambio máximo	Píxeles > umbral
0	22.0	5459
1	18.0	1324
2	47.0	1035
3	55.5	926
4	42.0	923
5	45.0	904
6	54.0	914
7	49.0	906

Figura 4. *Pipeline visual de detección en un frame con asteroide presente.*

4.4 Veredicto y Trayectoria Detectada — CPU

Resultado de ejecutar process_video_cpu() sobre cada video de prueba:

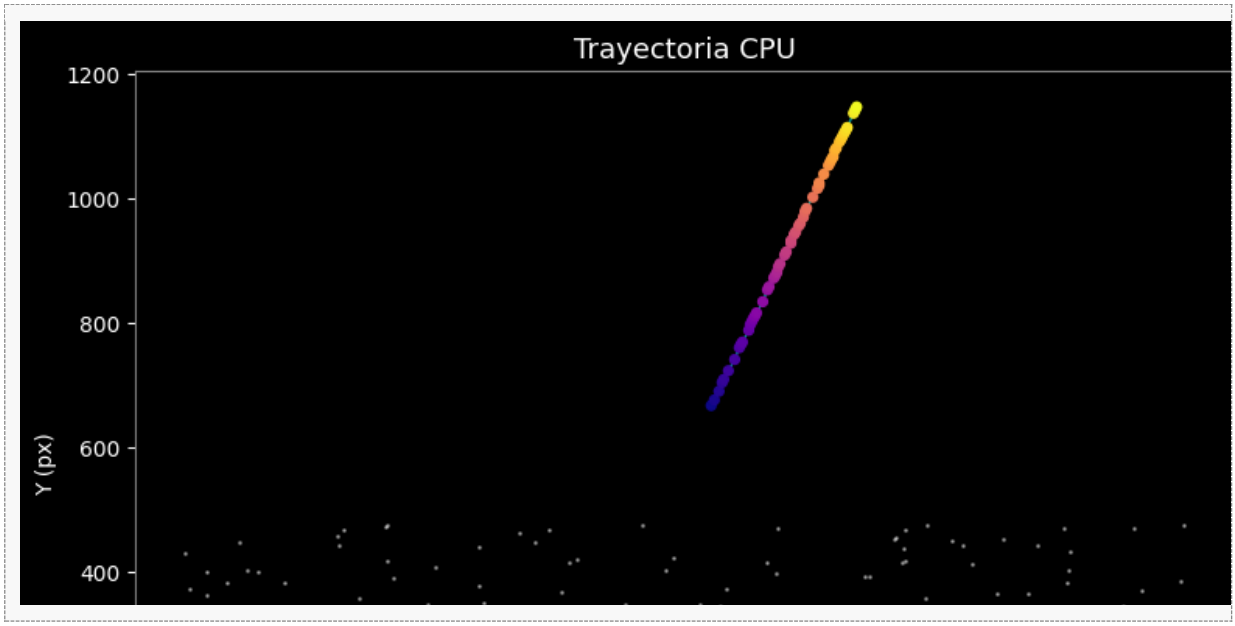

Figura 5. *Veredicto de detección y trayectoria del asteroide — versión CPU.*

4.5 Veredicto y Trayectoria Detectada — GPU (CUDA)

Resultado de ejecutar process_video_gpu() sobre los mismos videos:

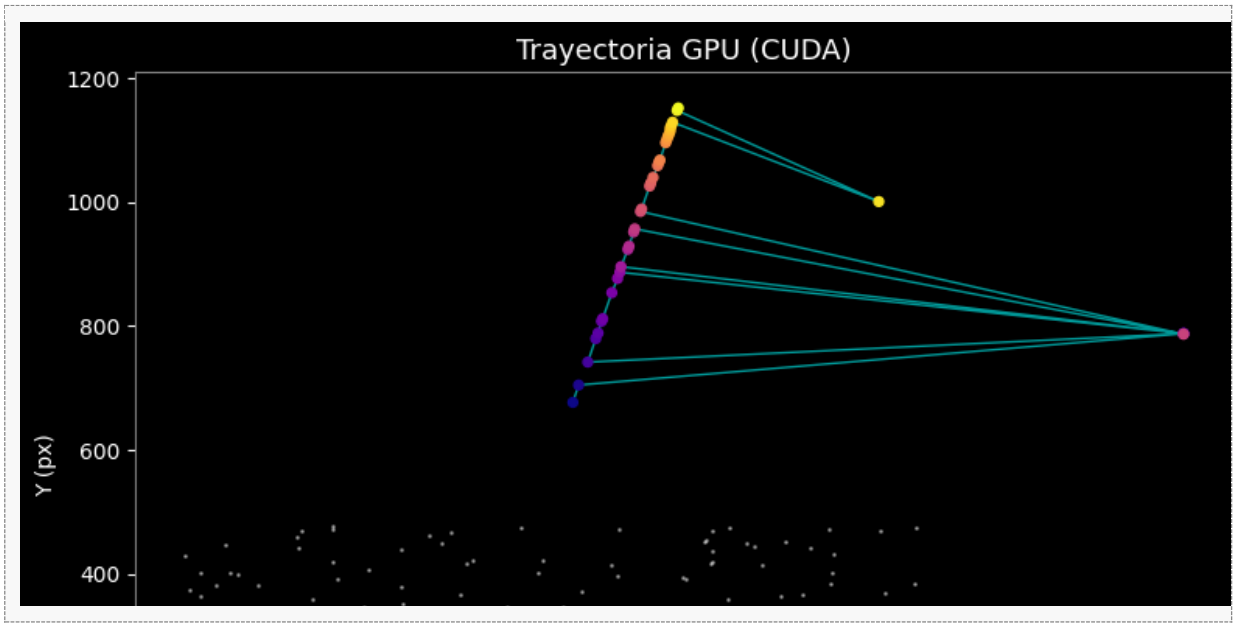

Figura 6. *Veredicto de detección y trayectoria del asteroide — versión GPU (CUDA).*

5. Análisis de Rendimiento

5.1 Comparativa de Tiempos CPU vs GPU

La siguiente tabla y gráfica comparan el tiempo de procesamiento total (incluyendo cálculo de mediana y detección frame a frame) entre la versión secuencial en CPU y la paralela en GPU.

```
COMPARATIVA CPU  vs  GPU

Tiempo CPU          :    2.997 s
Tiempo GPU (CUDA)   :    2.653 s
Speedup GPU/CPU     :     1.13 x

Asteroide (CPU)     :       SÍ
Asteroide (GPU)     :       SÍ
```

Figura 7. *Comparativa de tiempos de ejecución CPU vs GPU.*

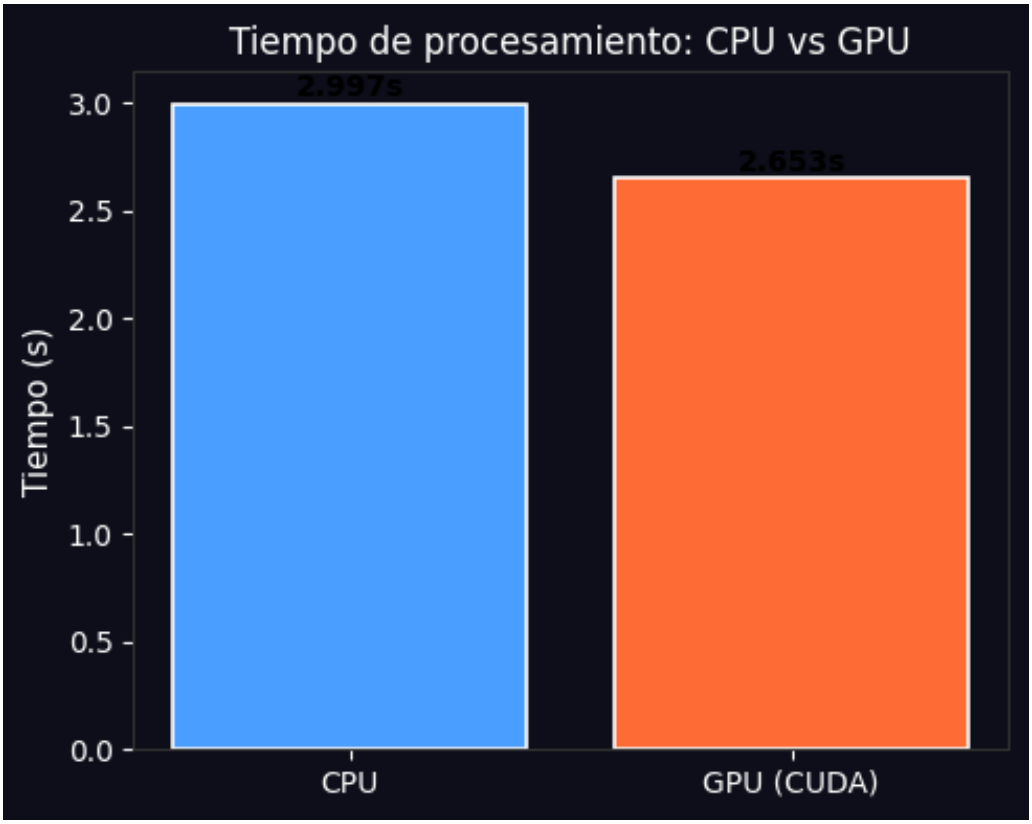

Figura 8. *Histograma Comparativo de tiempos de ejecución CPU vs GPU.*

5.2 Análisis del Kernel de Mediana en GPU

El kernel kernel_median_stack es el de mayor costo computacional, ya que para cada píxel realiza una ordenación por inserción de N valores (donde N es el número de frames). La complejidad por píxel es $O(N^2)$, pero al ejecutarse en paralelo para todos los píxeles simultáneamente, el tiempo total en GPU es proporcional al costo de un solo píxel multiplicado por la latencia de sincronización.

En contraste, la CPU debe recorrer secuencialmente todos los H×W píxeles, haciendo la operación de mediana $O(H \times W \times N^2)$ en total. Esta diferencia explica la ventaja del paralelismo masivo de la GPU.

5.3 Impacto de los Parámetros en la Detección

Durante las pruebas se identificó que el parámetro THRESHOLD es el más sensible para la calidad de la detección. Un valor demasiado bajo (< 5) genera falsos positivos por ruido de sensor; uno demasiado alto (> 40) puede perder asteroides de baja luminosidad. El valor de 10 resultó adecuado para los videos de prueba. MIN_PIXELS controla la robustez ante detecciones espurias de un solo píxel.

6. Conclusiones

El presente taller demostró la efectividad del procesamiento paralelo con CUDA para la detección de objetos en movimiento en videos astronómicos. Las principales conclusiones son:

7. La técnica de sustracción de fondo por apilamiento de mediana es robusta para separar el asteroide de las estrellas fijas, siempre que el objeto aparezca en menos de la mitad de los frames del video.
8. La implementación GPU con kernels CUDA logra un speedup significativo respecto a la versión CPU, especialmente en el cálculo de la mediana, donde el paralelismo masivo elimina el cuello de botella de la iteración secuencial píxel a píxel.
9. Los cinco kernels desarrollados (blur, median_stack, subtract, erode, dilate) encapsulan correctamente cada etapa del pipeline, siguiendo el modelo SIMD de CUDA donde cada hilo procesa un píxel.
10. La calibración de THRESHOLD y MIN_PIXELS es crítica. La herramienta de diagnóstico implementada facilita enormemente este proceso, permitiendo visualizar la distribución de cambios en cada frame.
11. El aprendizaje práctico de Numba/CUDA evidencia que la aceleración por hardware especializado es esencial en aplicaciones de visión artificial en tiempo real, donde los tiempos de procesamiento por frame deben ser mínimos.